



EXPRESSIONS AND ASSIGNMENT STATEMENT STATEMENTS

T: Ch. 7

Introduction

- Expressions are the fundamental means of specifying computations in a programming language.
- To understand expression evaluation, need to be familiar with the order of operators and operand evaluation.
- Arithmetic evaluation was one of the motivations for the development of the first programming languages.
- Arithmetic expressions consist of operators, operands, parentheses, and function calls
 - In most languages, binary operators are infix, such as $x + 6$
 - Scheme and LISP, in which they are prefix, such as $+ x 6$
 - Perl also has some prefix binary operators
 - Most unary operators are prefix, but the $++$ and $--$ operators in C-based languages can be either prefix or postfix

Arithmetic Expressions Issues

- Design issues for arithmetic expressions
 - Operator precedence
 - Operator associativity
 - Order of operand evaluation
 - Operand evaluation side effects
 - Operator overloading
 - Type mixing in expressions

Expressions

- Operators are:
 - A unary operator has one operand
 - A binary operator has two operands
 - A ternary operator has three operands
- Precedence rules
 - The *operator precedence rules* for expression evaluation define the order in which “adjacent” operators of different precedence levels are evaluated.
 - Typical precedence levels
 - parentheses
 - unary operators
 - ** (if the language supports it)
 - *, /
 - +, -

Expressions

- Operator Associativity

- The *operator associativity rules* for expression evaluation define the order in which adjacent operators with the same precedence level are evaluated.
- Typical associativity rules in C++
 - Left to right, except = assignment operator, which is right to left.
 - Sometimes unary operators associate right to left.

- Precedence and associativity rules can be overridden with parentheses.

Ternary Operator Example: Conditional Expressions

- Conditional Expressions

- C-based languages (e.g., C, C++)

- An example:

- ```
average = (count == 0)? 0 : sum / count
```

- Evaluates as if written as follows:

- ```
if (count == 0)
    average = 0
else
    average = sum / count
```

Comma Operator in C/C++

- Comma operator (C and C++)

- Evaluate the operands in sequence.
- The value of the expression is the value of the last operand.
- An example:

average = (a++, a+b);

- Evaluates as if it is written as follows:

a++; average = a + b;

- Useful in the for-statements to initialize multiple variables or increment multiple variables. Such as,
for (i = 0, j=1; i <= 10, j > 0; i++, j--) { . . . }

Expressions: Operand Evaluation Order

■ *Operand evaluation*

1. **Variables**: fetch the value from memory.
2. **Constants**: sometimes a fetch from memory; sometimes the constant is in the machine language instruction.
3. **Parenthesized expressions**: evaluate all operands and operators first.
4. The most interesting case is when an operand is a **function call**.

■ Precedence and associativity rules do not specify the order of evaluating operands of an operator in expressions.

■ If none of the operands of an operator has side effects, then operand evaluation order is irrelevant.

- ☐ The most interesting case is when an operand is a function call.
- ☐ Operators having side effects are the increment and decrement (`++`, `--`) postfix and prefix operators.

Expressions: Potentials for Side Effects

- **Functional side effects:** when a function changes a two-way parameter or a non-local variable.
 - If a function also changes a variable outside of the function (either some non-local variable, or some memory that it has a pointer or a reference to) then the function has a side effect.
 - **Side effects can be useful**
 - Example: the `get()` method on a stream returns the next character from the stream and, as a side effect, changes the read position in the stream.
 - **However, side effects might cause problems.**
- **Problem with functional side effects:**
 - Example

```
a = 10;
/* assume that fun changes its parameter */
b = a + fun(&a);
```
 - Without additional information, it is not known whether `a` is evaluated first or `f(a)` is evaluated first!

Functional Side Effects

```
int f(int& x) {  
    x = x + 5;  
    return x;  
}  
/////////////////  
int a = 1;  
int b;  
b = a + f(a);
```

- Evaluate a first
 - $1 + f(1)$
 - $1 + 6$
 - Result is 7, a is 6
- Evaluate f(a) first
 - f(1) returns 6
 - Value of a is 6
 - $6 + 6$
 - Result is 12, a is 6

- It is not clear whether the variable “a” is evaluated first or f(a) is evaluated first!

Functional Side Effects

- Two possible solutions to the problem
 1. Write the language definition to disallow functional side effects
 - No two-way parameters in functions
 - No non-local references in functions
 - **Advantage:** it works!
 - **Disadvantage:** inflexibility of one-way parameters and lack of non-local references
 2. Write the language definition to demand that operand evaluation order be fixed
 - **Disadvantage:** limits some compiler optimizations
 - Java requires that operands appear to be evaluated in left-to-right order
- When there is no fixed order for operand evaluations defined by the language, the compiler would choose the order which is likely to generate faster code.

Overloaded Operators

- Use of an operator for more than one purpose is called *operator overloading*
 - Some are common (e.g., + for **int** and **float**)
 - Some are potential trouble (e.g., * and & in C and C++)
 - * has been used as a binary operator for multiplication and as a unary operator for dereferencing pointer variable.
 - Similarly can be said about the ampersand operator &.
- C++, C#, and F# allow user-defined overloaded operators
 - When sensibly used, such operators can be an aid to readability (avoid method calls, expressions appear natural)
 - Potential problems:
 - Users can define nonsense operations (e.g., using + to mean multiplication).
 - Readability may suffer, even when the operators make sense.

Type Conversions

■ Conversions:

- A *narrowing conversion* is one that converts an object to a type that cannot include all of the values of the original type e.g., `float` to `int`.
- A *widening conversion* is one in which an object is converted to a type that can include at least approximations to all of the values of the original type e.g., `int` to `float`.

■ Mixed-Mode Expression

- A *mixed-mode expression* is one that has operands of different types.
- A *coercion* is an implicit type conversion
 - Computers do not have binary operations that take operands of different types.

Type Conversions

- Disadvantage of coercions:

- They decrease the type error detection ability of the compiler. Because of that mixed-mode expressions are not allowed by Ada, F# and ML.

- In most common languages all numeric types are coerced in expressions, using widening conversions.

- Explicit Type Conversions

- Called *casting* in C-based languages

- Examples

- C/C++: `(int) angle`

- Parentheses allow specifying C types of two-word names, such as (long int)

- C++/F# Also allow using the syntax of function calls for single-word type names. For example `int (angle)`

- For more details see C++ online documentations at https://en.cppreference.com/w/cpp/language/explicit_cast

- Some languages will check certain casts at runtime.

Expressions (Cont'd)

■ Errors in Expressions

□ Causes

- Inherent limitations of arithmetic, e.g., division by zero
- Limitations of computer arithmetic, e.g. overflow or underflow
 - The result of an operation cannot be represented in the memory cell where it must be stored.

□ Often ignored by the run-time system

- Might be handled with an “exception handling” mechanism
 - Exception handling will be discussed later on.

Relational and Boolean Expressions

■ Relational Expressions

- Relational expression has two operands and one relational operator.
- The result of a relational expression is Boolean (unless Boolean is not a type included in the language, such as in C-language).
- Operator symbols used vary somewhat among languages (`!=`, `/=`, `~=`, `.NE.`, `<>`, `#`)
- Relational operators are commonly overloaded for a variety of types.
 - Typical types of operands: numeric types, strings, and enumeration types
- JavaScript and PHP have two additional relational operator, `===` and `!==`
 - Similar to their cousins, `==` and `!=`, except that they do not coerce their operands.
 - Ruby uses `==` for equality relation operator that uses coercions and `eq?` for those that do not.

Relational and Boolean Expressions

■ Boolean Expressions

- Operands are Boolean (Boolean variables, constants, or relational expressions) and the result is Boolean.
- Example operators: And (&&), Or (||), Not (!), and sometimes exclusive OR (XOR) and equivalence.
- C-based languages (such as C/C++ and Java) assign a higher precedence level to AND (&&) than OR (||):
 - In math the two have the same precedence
- Precedence levels of arithmetic, relational, and Boolean operators in C-based languages
 - Highest: postfix ++, and --
 - Unary +, unary -, unary ! (NOT), prefix ++ and --
 - *, /, %
 - Binary +, binary -
 - <, >, <=, >=
 - =, !=
 - &&
 - Lowest: ||

Relational and Boolean Expressions

- Since original C-language (prior to C99) has no Boolean type, numeric values are used to represent Boolean values.
 - Zero value considered as false and all nonzero values as true.
 - Arithmetic expressions can be used for Boolean expressions in C99 and C++ (but not in Java).
- One odd characteristic of C's expressions:
 - **`a < b < c`** is a legal expression, but the result is not what you might expect:
 - Left operator is evaluated, producing 0 or 1.
 - The evaluation result is then compared with the third operand (i.e., **`c`**).

Short Circuit Evaluation

- An expression in which the result is determined without evaluating all of the operands and/or operators
- Example:
 - `(a >= 0) && (b < 10)`
 - The value of the Boolean expression is independent of the second relational expression if `(a < 0)` (i.e., value of first relational expression is false)
- Potential Problem with non-short-circuit evaluation
 - Example: a table lookup code using a while statement, assuming an array list of *listlen* elements.

```
index = 0;
while (index < listlen) && (LIST[index] != value)
    index++;
```
 - Since there is no short-circuit, when `index==listlen` (value is not found in the table), `LIST[index]` will cause an indexing error.

Short Circuit Evaluation (continued)

- Supporting short-circuit evaluations of Boolean expressions and also having potential side effects in expressions would cause errors to occur.
 - Example: `(a > b) || ((b++) / 3)`
 - The correctness of program depends on the side effect evaluation.
 - However, `b` will be changed only if `a <= b`
- C, C++, and Java: use short-circuit evaluation for the usual Boolean operators (`&&` and `||`), but also provide bitwise Boolean operators that are not short circuit (`&` and `|`).
 - Bitwise Boolean operators are equivalent to usual Boolean operators if all operands are restricted to either false or true.
- All logic operators in Ruby, Perl, ML, F#, and Python are short-circuit evaluated.

Assignment Statements

- The general syntax in C++

`<target_var> = <expression>`

- It is used to dynamically change the bindings of values to variables.

- The assignment operator

= Fortran, BASIC, the C-based languages

:= Ada, Pascal (Project language)

- = can be bad when it is overloaded for the relational operator for equality (that's why the C-based languages use == as the relational operator)

- Compound Assignment Operators

- A shorthand method of specifying a commonly needed form of assignment.

- Introduced in ALGOL; adopted by C and the C-based languages.

- Example: `a = a + b`, can be written as `a += b`

Assignment Statements

■ Unary Assignment Operators

- Unary assignment operators in C-based languages combine increment and decrement operations with assignment.

- Prefix and postfix ++ and -- operators

- Examples

`sum = ++count; // (count incremented, then assigned to sum)`

Equivalent to: `count = count + 1; sum = count;`

`sum = count++; // (count assigned to sum, then incremented)`

Equivalent to: `sum = count; count = count + 1;`

- `-count++` (count incremented then negated, postfix ++ is having higher precedence than unary ops for signs):

- Equivalen to: `- (count ++)`

- Prefix ++, -- have the same precedence of unary + and – in C-Based languages

Assignment

■ Assignment as an Expression

- In the C-based languages, Perl, and JavaScript, the assignment statement produces a result which is the same as the value assigned to the target. It can be used as an expression and as an operand in other expressions.

```
while ( (ch = getchar()) != EOF ) { ... }
```

- (ch = getchar()) is carried out; the result (assigned to ch) is used as an operand to be compared with the EOF constant.
- Example: a = b = c = 10; is valid
 - Assignment associates right to left.
- Disadvantage: another kind of expression side effect.

■ Mixed-Mode Assignment

- Assignment statements can also be mixed-mode.
- In C++, any numeric type value can be assigned to any numeric type variable.

Mixed-Mode Assignment

- Assignment statements can also be mixed-mode.
 - In Fortran, C, Perl, and C++, any numeric type value can be assigned to any numeric type variable.
 - In Java and C#, only widening assignment coercions are done.
 - **In Ada, there is no assignment coercion.**
 - **In Pascal (*and Basic Pascal-Like project language*): only widening assignment coercions are done.**

Basic Pascal-Like Language Definition of Expressions and Assignment Statement

- `AssignStmt ::= Variable := Expr`
- `Variable ::= IDENT`
- `ReadLnStmt ::= ReadLnStmt ( VarList )`
- `ExprList ::= Expr { , Expr }`
- `VarList ::= Variable { , Variable }`
- `Expr ::= RelExpr ::= SimpleExpr [ ( = | < | > ) SimpleExpr ]`
- `SimpleExpr ::= Term { ( + | - | OR ) Term }`
- `Term ::= SFactor { ( * | / | DIV | MOD | AND ) SFactor }`
- `SFactor ::= [ ( - | + | NOT ) ] Factor`
- `Factor ::= IDENT | ICONST | RCONST | SCONST | BCONST | CCONST | (Expr)`

Basic Pascal-Like Language: Expressions and Assignment Statement

Table of Associativity and Precedence Levels of Operators

Precedence	Operator	Description	Associativity
1	Unary +, -, not	Unary plus and minus and NOT	Right-to-Left
2	*, /, div, mod, and	Multiplication, Division, integer division, modulus, and logical AND	Left-to-Right
3	+, -, or	Addition, Subtraction, and logical OR	Left-to-Right
4	=, <, >	Equality, and Relational operators less than, and greater than	Left-to-Right

Basic Pascal-Like Language: Expressions and Assignment Statement

- The ASSOP operator ($:=$) in the *AssignStmt* assigns a value to a variable. It evaluates the *Expr* on the right-hand side of the operator and saves the expression value in a memory location associated with the left-hand side variable (*Variable*). A left-hand side variable of an integer type must be assigned an integer value. However, a real type variable can be assigned real or integer values. A string type variable can be assigned a string or character type values. While a left-hand side variable of character or Boolean type must be assigned a value of the same type as of the left-hand side variable. Note that, it is illegal to assign a null string to a character variable.

Basic Pascal-Like Language: Expressions and Assignment Statement

- The binary operations of numeric operators as addition, subtraction, multiplication, division, integer division, and modulus are performed upon two numeric operands. While the binary logic operators are performed upon two Boolean operands. Mixed-mode numeric operand types of integer and real are allowed for addition, subtraction, multiplication and division, where the integer operand value is converted (coerced) into a real value for the operation. The division operation (DIV) produces a real result regardless of the types of the division operands. However, the operands' types for integer division (IDIV) and modulus operation (MOD) should be integer, and the result is an integer. The result of the modulus operation ($i \bmod j$) is the remainder of the integer division of i by j . The sign of the result of a MOD operator is the same as the sign of the left side operand of the MOD operator.

Basic Pascal-Like Language: Expressions and Assignment Statement

- The supported string operations are the concatenation, using the PLUS operator, and string trimming using the minus sign. The PLUS is treated as an overloaded binary operator for concatenation applied on operands of string or character types. The trimming operation is a unary operation that removes the trailing blanks in a string or a character operand. The minus sign is treated as an overloaded operation that is applied on a string or character operand type.
- Similarly, relational operators ($=$, $<$, and $>$) operate upon two compatible type operands. The evaluation of a relational expression produces either a logical true or false value. For all relational operators, no cascading is allowed.
- The unary sign operators (+ or -) are applied upon unary numeric type operands only. While the unary NOT operator is applied upon a Boolean type operand only.



Summary

- Expressions
- Operator precedence and associativity
- Operator overloading
- Mixed-type expressions
- Various forms of assignment